

Provider Config Reference

Provider Config Reference · docs/latest · generated 2026-08-15

Generated from the types — field tables are extracted from `packages/types/shared-types/src/dataProvider.ts` by `scripts/gen-provider-config-reference.mjs`. Edit the type doc comments (or this script's prose), then re-run `node scripts/gen-provider-config-reference.mjs`.

A **data provider** is a persisted catalog row that tells the SharedWorker data hub how to reach a data source. The row wrapper is `DataProviderConfig`; its `config` field is a `TransportConfig` — a **discriminated union over `providerType`** with six variants. There is no base transport interface; each variant is complete on its own (the two SSRM variants extend their streaming siblings).

Ways a row gets created:

- `createStarui({ providers })` — create-if-missing seeding (deterministic `providerId` required; later editor edits survive reloads).
- The **Data Provider Editor** (`npm run app -- dataprovider-editor`, or the in-grid dialog) — validates on save with `validateProviderConfig`.
- A **deploy seed** (`seed.json`) — see Seed formats.

Provider types

PROVIDERTYPE	ROW MODEL	WHAT IT IS
<code>stomp</code>	CSRM	STOMP-over-WebSocket streaming: snapshot then live deltas, whole dataset to the client
<code>stomp-ssrm</code>	SSRM	same wire, plus the SharedWorker attaches a server-side-row-model query plane — grids page blocks
<code>rest</code>	CSRM	one-shot HTTP fetch — no live updates
<code>mock</code>	CSRM	synthetic rows with optional periodic updates (labs / offline)
<code>mock-ssrm</code>	SSRM	synthetic rows behind the SSRM query plane
<code>appdata</code>	—	not a stream: a key/value bag other configs reference via <code>{{name.key}}</code> — see AppData →

The **CSRM-vs-SSRM discriminator** is `isSsrmProviderType(type)` (`stomp-ssrm` or `mock-ssrm`). Every mode decision routes through that one predicate — never test the string suffix inline.

DataProviderConfig — the catalog row wrapper

DataProvider configuration wrapper

FIELD	TYPE		NOTES
providerId	string	optional	
name	string	required	
description	string	optional	
providerType	ProviderType	required	
config	TransportConfig	required	
tags	string[]	optional	
isDefault	boolean	optional	
userId	string	required	
public	boolean	optional	Visibility: true → row is saved with userId='system' (visible to everyone sharing the appld). false / undefined → row is saved with the active userId (visible only to the author). The configurator surfaces this as a single "Public" toggle.

Storage mapping (DataProviderConfigStore): configId = providerId, componentType: 'data-provider', componentSubType = providerType, displayText = name, payload = config + __providerMeta. Provider rows are **platform-global**: every user sees the catalog; public: true rows are owned by 'system'.

StompProviderConfig (providerType: 'stomp')

STOMP Provider Configuration

FIELD	TYPE		NOTES
<code>providerType</code>	<code>'stomp'</code>	required	
<code>websocketUrl</code>	<code>string</code>	required	
<code>listenerTopic</code>	<code>string</code>	required	
<code>requestMessage</code>	<code>string</code>	optional	
<code>requestBody</code>	<code>string</code>	optional	
<code>snapshotEndToken</code>	<code>string</code>	optional	
<code>heartbeat</code>	<code>{ outgoing?: number; incoming?: number; }</code>	optional	
<code>keyColumn</code>	<code>string readonly string[]</code>	optional	Unique-row identity. A SINGLE column name keys rows by that one field; an array of column names keys rows by the joined values (separator: <code>-</code>) — used for datasets with composite primary keys. Drives both the worker-side cache (Hub) and AG-Grid's <code>getRowId</code> .

FIELD	TYPE		NOTES
<code>inferredFields</code>	<code>FieldInfo[]</code>	optional	
<code>columnDefinitions</code>	<code>ColumnDefinition[]</code>	optional	
<code>conflateEnabled</code>	<code>boolean</code>	optional	Master on/off for live-update conflation. Defaults to ON (<code>undefined</code> / <code>true</code>). Set <code>false</code> to deliver every live row update even when <code>conflateByKey</code> / <code>keyColumn</code> would otherwise supply a conflation key. This is the explicit disable switch: without it, conflation falls back to <code>keyColumn</code> and can't be turned off independently of throttling.
<code>conflateByKey</code>	<code>string</code>	optional	Conflate row updates by this column before fanning out to subscribers. Two updates for the same key value within a <code>throttleMs</code> window collapse into the latest one (upsert semantics). Typically set to the same value as <code>keyColumn</code> so grids see exactly one update per row per flush. When unset, conflation falls back to <code>keyColumn</code> ; set <code>conflateEnabled: false</code> to turn conflation off entirely.
<code>throttleEnabled</code>	<code>boolean</code>	optional	Master on/off for live-update throttling. Defaults to ON (<code>undefined</code> / <code>true</code>). Set <code>false</code> to fan out every live delta immediately even when <code>throttleMs</code> is set — the ms value is kept so re-enabling restores the previous window.
<code>throttleMs</code>	<code>number</code>	optional	Coalesce row-update fanout into trailing-edge bursts every <code>throttleMs</code> . 0 / <code>undefined</code> → immediate fanout (no batching). The conflation window above only takes effect when this is set and <code>throttleEnabled</code> is not <code>false</code> .
<code>snapshotChunkSize</code>	<code>number</code>	optional	Max rows shipped per <code>postMessage</code> when flushing the snapshot from the worker to the client. Larger snapshots split into this many rows per replace/delta frame so each main-thread <code>message</code> deserialize stays under Chromium's ~50ms long-task budget. Default 500. Settable in code (author the config) or via the provider editor.
<code>projectFields</code>	<code>boolean</code>	optional	Prune incoming rows to the fields the UI can actually see — the <code>columnDefinitions[].field</code> paths plus <code>keyColumn</code> — at frame-parse time in the worker, BEFORE rows enter the snapshot buffer / hub cache. Upstream feeds that ship wide objects (e.g. 2000 fields when the blotter renders 200) otherwise pay ~10x on worker memory, snapshot encode, <code>postMessage</code> payloads and client parse in every window. Nested paths (<code>a.b.c</code>) copy just the needed subtree. Default OFF. Changing the visible fields requires a provider restart (the editor's Restart already rebuilds the slot from the new cfg).

FIELD	TYPE		NOTES
<code>thinDeltas</code>	<code>boolean</code>	optional	Thin field-level deltas. When ON, post-ready live updates broadcast only the top-level fields that actually changed per row (<code>delta-patch</code> wire events) instead of full replacement rows — touch updates that change a few fields out of hundreds shrink the hub→window wire by the touch ratio. The client merges each patch into its previous full row producing a NEW row object, so subscribers still observe whole immutable rows. Requires <code>keyColumn</code> (ignored without it). Snapshot/replace frames always ship full rows. Default OFF.
<code>wireFormat</code>	<code>'json' 'columnar'</code>	optional	Wire codec for binary hub→window frames (snapshot replay, restart broadcast, large live batches). - <code>'columnar'</code> (default — see <code>STOMP_TUNING_DEFAULTS.wireFormat</code> ; the hub runs columnar unless <code>'json'</code> is set explicitly) — typed-array columnar frames: numbers travel as raw Float64 and booleans as bitmaps, cutting each window's per-frame decode several-fold on number-heavy feeds. Frames that don't qualify (non-object rows) fall back to JSON per-chunk automatically. - <code>'json'</code> — UTF-8 <code>JSON.stringify</code> bytes, decoded with <code>JSON.parse</code> .
<code>reconnect</code>	<pre>{ /** Static reconnect delay in ms. Default 5000 (matches prior behaviour). */ initialDelayMs?: number; }</pre>	optional	Reconnect policy — <code>initialDelayMs</code> becomes the stompjs <code>reconnectDelay</code> . Default 5000.

Effective runtime defaults — `STOMP_TUNING_DEFAULTS`

The values the worker applies when a tuning field is unset — single-sourced so the transport, the hub, and the editor's placeholder text can never disagree:

KNOB	EFFECTIVE DEFAULT	WHAT IT IS
<code>throttleMs</code>	<code>25</code>	Trailing-edge live fan-out window (ms).
<code>snapshotChunkSize</code>	<code>500</code>	Rows per snapshot postMessage chunk.
<code>reconnectInitialDelayMs</code>	<code>5000</code>	stompjs <code>reconnectDelay</code> (ms).
<code>heartbeatMs</code>	<code>4000</code>	Heartbeat interval, both directions (ms).
<code>wireFormat</code>	<code>'columnar'</code>	Binary hub→window codec — columnar unless explicitly 'json'.
<code>publishWindowMs</code>	<code>0</code>	SSRM publish window (ms) — 0 = flush per tick.

Boolean knobs resolve in the transport as `cfg.conflateEnabled !== false` / `cfg.throttleEnabled !== false` — both default **ON**; only an explicit `false` disables.

StompSsrnProviderConfig (providerType: 'stomp-ssrm')

STOMP + SSRM Provider Configuration. Same wire/transport as {@link StompProviderConfig}; the SharedWorker attaches an SSRM query plane so grids use `rowModelType: 'serverSide'`.

Extends `Omit<StompProviderConfig, 'providerType'>` — every base field applies too.

FIELD	TYPE		NOTES
<code>providerType</code>	<code>'stomp-ssrm'</code>	required	
<code>blockSize</code>	number	optional	Hint for AG Grid <code>cacheBlockSize</code> (client); worker pages by request.
<code>publishWindowMs</code>	number	optional	Trailing-edge SSRM flush window in ms; 0/omitted = per-frame passthrough. Accumulates and key-conflates changed rows across the window before the worker fans a single <code>ssrm-tick</code> — see <code>SsrnServer</code> / <code>SsrnPlane</code> in <code>@wellsfargo-starui/data</code> .

RestProviderConfig (providerType: 'rest')

REST Provider Configuration

FIELD	TYPE		NOTES
<code>providerType</code>	<code>'rest'</code>	required	
<code>baseUrl</code>	string	required	
<code>endpoint</code>	string	required	
<code>method</code>	<code>'GET' 'POST'</code>	required	

FIELD	TYPE		NOTES
<code>queryParams</code>	<code>Record<string, string></code>	optional	
<code>body</code>	<code>string</code>	optional	
<code>headers</code>	<code>Record<string, string></code>	optional	
<code>pollInterval</code>	<code>number</code>	optional	⚠ declared but not read by the REST transport today (only <code>validateProviderConfig</code> warns on it).
<code>paginationMode</code>	<code>'offset' 'cursor' 'page'</code>	optional	⚠ declared but not consumed by any runtime code today.
<code>pageSize</code>	<code>number</code>	optional	⚠ declared but not consumed by any runtime code today.
<code>auth</code>	<code>{ type: 'bearer' 'apikey' 'basic'; credentials: string; headerName?: string; }</code>	optional	
<code>timeout</code>	<code>number</code>	optional	⚠ declared but not read by the REST transport today.
<code>keyColumn</code>	<code>string readonly string[]</code>	optional	Required for streaming consumers (MarketsGrid). The column whose value uniquely identifies a row — drives RowCache upsert + AG-Grid <code>getRowId</code> . Accepts an array of column names for composite keys (joined with <code>-</code>).
<code>rowsPath</code>	<code>string</code>	optional	Path inside the JSON response that holds the rows array. Dot notation; e.g. <code>data.results</code> . Default: response is the rows array directly.
<code>inferredFields</code>	<code>FieldInfo[]</code>	optional	Persisted schema introspection — see <code>StompProviderConfig</code> .
<code>columnDefinitions</code>	<code>ColumnDefinition[]</code>	optional	
<code>conflateByKey</code>	<code>string</code>	optional	See <code>StompProviderConfig</code> — same fanout knobs apply.

FIELD	TYPE		NOTES
<code>throttleMs</code>	<code>number</code>	optional	

MockProviderConfig (providerType: 'mock')

Mock Provider Configuration

FIELD	TYPE		NOTES
<code>providerType</code>	<code>'mock'</code>	required	
<code>dataType</code>	<code>'positions' 'trades' 'orders' 'custom'</code>	required	
<code>updateInterval</code>	<code>number</code>	optional	
<code>updateIntervalMs</code>	<code>number</code>	optional	Alias for <code>updateInterval</code> in ms — preserved for UI template compatibility.
<code>rowCount</code>	<code>number</code>	optional	
<code>enableUpdates</code>	<code>boolean</code>	optional	
<code>customData</code>	<code>any[]</code>	optional	

FIELD	TYPE		NOTES
keyColumn	string readonly string[]	optional	Unique-row identity, same semantics as the other streaming configs. Required when this cfg is attached through the SharedWorker hub (<code>useProviderStream / client.attach</code>) — the hub keys its row cache by <code>keyColumn</code> and silently drops rows that don't resolve a value, so a missing field surfaces as an empty grid. Safe to omit when calling <code>startMock</code> directly in-process (the in-process path doesn't go through the cache). Typical values per dataType: <code>'cusip'</code> for positions, <code>'tradeId'</code> for trades, <code>'id'</code> for orders.

Update-interval resolution in the transport: `updateIntervalMs ?? updateInterval ?? per-dataType default`.

MockSsrProviderConfig (providerType: 'mock-ssrm')

Mock + SSRM Provider Configuration. Same row generator as {@link MockProviderConfig}; the SharedWorker attaches an SSRM query plane so grids use `rowModelType: 'serverSide'` with lab-parity field names (`id`, `cusip`, `bidPrice`, ...).

Extends `Omit<MockProviderConfig, 'providerType'>` — every base field applies too.

FIELD	TYPE		NOTES
providerType	'mock-ssrm'	required	
blockSize	number	optional	Hint for AG Grid <code>cacheBlockSize</code> (client); worker pages by request.
columnDefinitions	ColumnDefinition[]	optional	Optional persisted column schema for SSRM grids / editors.
publishWindowMs	number	optional	Trailing-edge SSRM flush window in ms; 0/omitted = per-frame passthrough. Accumulates and key-conflates changed rows across the window before the worker fans a single <code>ssrm-tick</code> — see <code>SsrServer / SsrPlane</code> in <code>@wellsfargo-starui/data</code> .

AppDataProviderConfig (providerType: 'appdata')

AppData Provider Configuration

FIELD	TYPE		NOTES
providerType	'appdata'	required	

FIELD	TYPE		NOTES
variables	Record<string, AppDataVariable>	required	

AppDataVariable

FIELD	TYPE		NOTES
key	string	required	
value	string number boolean object	required	
type	'string' 'number' 'boolean' 'json'	required	
description	string	optional	
sensitive	boolean	optional	

The AppData layer (mirror, hooks, `{{name.key}}` template resolution) has its own page: [AppData](#) →.

ColumnDefinition — persisted column schema

Column definition for AG-Grid

FIELD	TYPE		NOTES
field	string	required	
headerName	string	required	
cellDataTypes	'text' 'number' 'boolean' 'date' 'dateString' 'object'	optional	
width	number	optional	

FIELD	TYPE		NOTES
<code>filter</code>	<code>string boolean</code>	optional	
<code>sortable</code>	<code>boolean</code>	optional	
<code>resizable</code>	<code>boolean</code>	optional	
<code>hide</code>	<code>boolean</code>	optional	
<code>type</code>	<code>string</code>	optional	
<code>valueFormatter</code>	<code>string</code>	optional	
<code>cellRenderer</code>	<code>string</code>	optional	
<code>valueGetter</code>	<code>string</code>	optional	Optional DSL expression compiled to an AG-Grid <code>valueGetter</code> at runtime (via <code>@wellsfargo-starui/core</code> 's ExpressionEngine). Column refs use bracket syntax — <code>[cusip]</code> , <code>[a.b.c]</code> for nested, optional-chaining paths — e.g. <code>STARTS_WITH([cusip], "SPCL") AND [inventoryName] == null ? [pnl.wrapper.rdiInventoryName] : [inventoryName]</code> Empty / absent means no override (column falls back to its <code>field</code> binding or the default nested-path getter). The expression never throws at runtime: parse/eval failures fall back to the field value.

`ColumnDefinition` is a deliberately narrow, serializable subset of AG Grid's `ColDef` — the full per-column customization state (styles, formatters, templates) lives in the grid customizer's profile state, not on the provider.

Validation — `validateProviderConfig`

Wired into the provider editor's **save** and **JSON-import** paths. Hard errors mirror what the transports actually require at attach time:

APPLIES TO	CONDITION	SEVERITY	MESSAGE
all	<code>providerType</code> missing	error	Provider type is required
<code>stomp</code> , <code>stomp-ssrm</code>	<code>websocketUrl</code> missing/blank	error	WebSocket URL is required for STOMP providers
<code>stomp</code> , <code>stomp-ssrm</code>	URL present but not <code>ws://</code> / <code>wss://</code>	warn	WebSocket URL should typically start with <code>ws://</code> or <code>wss://</code>
<code>stomp</code> , <code>stomp-ssrm</code>	<code>listenerTopic</code> missing/blank	error	Listener topic is required for STOMP providers
<code>rest</code>	<code>baseUrl</code> missing/blank	error	Base URL is required for REST providers
<code>rest</code>	URL present but not <code>http://</code> / <code>https://</code>	warn	Base URL should typically start with <code>http://</code> or <code>https://</code>
<code>rest</code>	<code>endpoint</code> missing/blank	error	Endpoint is required for REST providers
<code>rest</code>	<code>pollInterval</code> < 1000	warn	Poll interval is very low (< 1 second), may cause high server load
<code>mock</code> , <code>mock-ssrm</code> , <code>appdata</code>	—	(no per-type rules)	

Returns `{ isValid, errors, warnings? }` — `warnings` is `undefined` (not `[]`) when clean.

Seed formats

Two JSON envelopes exist and are **deliberately incompatible**:

- **Provider export** (`kind: 'starui.dataProvider'`, version 1) — the editor's per-provider Export/Import round-trip. Strips `providerId`, `userId`, `isDefault`; import mints a fresh row.
- **Deploy seed** (`seed.json`) — a Config Browser "Export ALL" bundle (`SeedData: activeAppId`, `activeUserId`, `appRegistry`, `userProfiles`, `roles`, `permissions`, `appConfig?`). Applied only against an empty database (`ConfigManager.seedIfEmpty`). `parseSeedJson` **rejects** a provider export dropped in as `seed.json` with a pointed error.

Editor seed values (`DEFAULT_PROVIDER_CONFIGS`) differ from the runtime defaults above — they are the editor's starting form values, not what the worker applies to an unset field.